RootCause: AI-powered diagnostics assistant

1. Problem Definition

When backend services fail, software engineering teams often struggle to identify and resolve the root cause in a timely manner. Diagnosing a single failure typically requires searching through logs, technical documentation, past incident reports, and the application's source code to understand and fix the issue.

This manual process increases resolution time because engineers must search through scattered documentation, analyze error logs, and navigate large codebases to identify the root cause and determine where and how to implement a fix.

Our goal is to solve these diagnostic bottlenecks by building an on-demand, user-triggered AI assistant.

2. State of the Art

Currently, teams rely on three primary tools to assist with incident troubleshooting, each carrying distinct trade-offs:

Approach / Solution	Pros	Cons
1. General-Purpose Commercial LLMs *(e.g., pasting logs into ChatGPT or Claude)*	• Zero setup friction. • Strong general coding knowledge and syntax familiarity.	• **Zero internal context:** Blind to proprietary architecture and internal runbooks. • **Privacy concerns:** Pasting sensitive production logs into third-party APIs violates enterprise compliance.
2. Enterprise AIOps Platforms *(e.g., Datadog Bits AI, New Relic AI)*	• Deep integration with live telemetry. • Automated anomaly correlation.	• **Heavyweight & intrusive:** Requires continuous, real-time agent instrumentation across all servers. • **Prohibitive cost:** Vendor lock-in and expensive per-host licensing.
3. Naive RAG Chatbot	• Grounds responses in internal documentation. • Easy to prototype.	• **Passive retrieval only:** Cannot actively execute diagnostic tools (e.g., checking a status code or querying a config rule) to verify a hypothesis before answering.

3. Proposal

We propose **RootCause**, an on-demand diagnostic agent powered by a fine-tuned Small Language Model (SLM) that combines Retrieval-Augmented Generation (RAG) with active tool execution via the Model Context Protocol (MCP).

How It Works

1. **User Input:** The developer submits an error log, exception trace, or description of unexpected behavior.
2. **Information Retrieval:** RootCause searches internal documentation, past incident reports, and known solutions for similar problems.
3. **Error Investigation:** The AI analyzes the error and uses diagnostic tools to inspect the source code, check configuration settings, and identify potential causes.
4. **Diagnosis and Solution:** RootCause combines the collected information to explain the likely cause of the failure and suggest a fix, supported by relevant evidence.

What Makes RootCause Different

- **On-Demand Diagnosis:** Unlike continuous monitoring systems, RootCause runs only when requested, reducing unnecessary resource usage.
- **Active Investigation:** Instead of only retrieving information, it examines the relevant code and configuration to support its diagnosis.
- **Specialized AI Model:** A fine-tuned SLM is adapted to backend debugging tasks, aiming to provide consistent, structured, and relevant answers while allowing the system to run locally without relying on external AI APIs.